
Design Thinking Fundamentals

On your first day working with any software system the architecture is always TBD—to be discovered. Whether we start from a blank page or must uncover structures in an existing software system, the architecture we need is out there, somewhere, waiting for us to discover it. To design a software system's architecture we explore solutions at the same time we're working to uncover the problem to be solved.

To help you perform this challenging task, you'll learn a creative and analytical approach to problem solving that puts humans at the center of attention called *design thinking*. Focusing on the people affected by your design decisions helps you concentrate on the exact problems that must be solved. It also grounds your solution exploration by reminding you that your purpose is to build software that helps people.

In this chapter, you'll learn how to apply design thinking to software architecture. You'll start by learning the core principles of design thinking. Next, you'll learn how to use different design mindsets to keep your architecture moving forward in (mostly) the right direction. Finally, you'll see an approach for picking design mindsets.

The Four Principles of Design Thinking

Design thinking is less a process and more a way of thinking about problems and solutions from the perspective of the people affected by them. While design thinking is not a process, there are still rules to guide our design activities. In [*Design Thinking: Understand - Improve - Apply \(Understanding Innovation\) \[PML10\]*](#) Christoph Meinel and Larry Leifer propose four universal principles of design. These principles apply to software architecture as well as to detailed program design, user interaction design, or any other design-focused discipline.

Here are the four principles of design:

1. *Human rule*. All design is social in nature.
2. *Ambiguity rule*. Preserve ambiguity.
3. *Redesign rule*. All design is redesign.
4. *Tangibility rule*. Make ideas tangible to facilitate communication.

We'll use the acronym HART to help remember these principles. Let's examine the HART principles as they relate to software architecture design so we can see how to apply design thinking in the context of software architecture.

Design for Humans

Design is an inherently human-focused endeavor. We design software *for* people. We design software *with* people. Every design decision in the architecture helps individuals in some way. Every design decision must be understood by and shared with other humans.

Architects must empathize with all stakeholders. We care about end users as much as the people the end users help, the programmers who write the code, the testers who verify it, and even the managers who keep tabs on the development schedule. As we design a software system, we'll collaborate with the other humans on our team and show them respect by listening, assuming positive intent, and using human-centered design methods.

The Human rule also reminds us that architects are not separated from our teams. We work directly with them to design the architecture together. Building software is an intensely social activity. The idea of an *ivory tower architect* who designs the architecture isolated from the team is a myth. Software architects are an integral part of every team. Separating the architect from the team severs the human connection the architect shares with everyone touched by the architecture.

Empathizing with the humans who directly and indirectly interact with the architecture makes us a better designer, communicator, and leader.

Preserve Ambiguity

Ambiguity in engineering is dangerous. Once we've made a design decision, we must share it with precision and clarity. Allowing requirements, design decisions, and commitments to remain ambiguous is the best way to destroy a project. Before we solidify a design decision, we can use ambiguity to keep options open.

Since the goal of software architecture is to arrange structures that promote desired quality attributes, we'll focus our attention there. In [Less is more with minimalist architecture \[MB02\]](#), Ruth Malan and Dana Bredemeyer suggest architects design a *minimalist architecture*. A minimalist architecture only shows how high-priority quality attributes are achieved and reduces risks for promoting those quality attributes. All other design decisions are left open for downstream designers to determine.

Architecture minimalism implies that we want to defer binding design decisions for as long as responsible. Design decisions that do not directly influence a quality attribute or reduce risks threatening our ability to deliver software are more about detailed design than architecture. Such decisions can safely be left open for downstream designers to settle outside the architecture. You'll learn more about preserving ambiguity in [Design for Change, on page 75](#).

Preserving ambiguity allows us to deliver software even as the world around us changes.

Design Is Redesign

In [A Pattern Language: Towns, Buildings, Construction \[AISJ77\]](#), Christopher Alexander and others cataloged over 253 civil engineering problems with known good solutions. Topics ranged from construction materials to community organization techniques to building architectures. If you've ever enjoyed a perfect spring morning while sipping coffee at a sidewalk cafe, then you can thank Christopher Alexander for documenting the *sidewalk cafe* pattern as a *community building* solution.

The redesign rule encourages us to look to think about what we already know by exploring patterns and past designs. As time goes on and as we build more software, our institutional knowledge about how to design great software improves. Other teams have probably seen a problem similar to the one you face currently. Hopefully, someone documented a pattern you can use as a starting point for your architecture. Maybe someone built a framework designed to solve your exact problem?

When designing software architectures, we'll spend more time refining existing designs than we'll be creating new ones. One of the least effective ways to design software architecture is to ignore the software systems that came before us.

Make the Architecture Tangible

While the structures in the architecture can exist in code, this does not make the architecture any more tangible. Code is difficult to read and does not

make discussions about quality attributes, coarse-grained components, design rationale, or the consequences of our decisions any easier. If we want to share an architecture with others, then we need to make it real in a way code by itself will not allow.

There are many ways to make architecture tangible. Draw it. Make it come alive in the code you write. Build prototypes that let people experience structures and quality attributes. Create simple models that show how some part of the architecture works. Create relatable metaphors. Act out parts of the control flow of the system.

The tangibility rule is closely related to the Human rule. Humans must be able to relate to ideas to internalize them. The only way to share an architecture is to make it tangible.

The HART principles form the philosophical basis of our architecture design approach. These principles guide our decision making and permeate our thinking. These four principles describe why we do things the way we do them. Now that you understand the underlying principles behind design thinking, let's explore how to apply these principles by learning how to select architecture-focused design practices.

Adopt a Design Mindset

Designing a software system requires us to think about the architecture from the perspective of different *design mindsets*. A design mindset is a way of thinking about the world so that we focus our attention on the right details at the right time.

There are four design mindsets: understand, explore, make, and evaluate. Each design mindset comes with a set of practices. To design the architecture, we'll choose a mindset, pick a practice in that mindset, apply the practice to learn something new about the architecture, and repeat.

The chapters in Part II will show you how to put each mindset into practice. Look for the icon, shown in the [image on page 19](#), at the start of each chapter to tell you what the focus will be. For now, let's learn what it means to embrace each of the four design mindsets.

Understand the Problem

In the *understand mindset* we actively seek information from stakeholders and work to describe the problem. The understand mindset is as much about requirements as it is empathy. To understand the problem, we must learn about the people who will be touched by our system and what they need.

Four Design Mindsets; Use in Any Order



To understand the problem, we'll need to investigate business goals and quality attributes that are important to our stakeholders. We'll also have to learn how our team operates and get a deeper sense of the priorities and trade-offs among design decisions.

Explore Ideas

In the popular ethos, design thinking is all about brainstorming and sticky notes. Brainstorming is powerful, but it is only one practice in the *explore mindset*. When we explore, we create multiple design concepts and identify engineering approaches for solving some aspect of a problem.

Exploring software architecture means we try combinations of structures until we find a combination that best promotes desired quality attributes. To find the best mix of structures, we'll need to survey a broad range of patterns, technologies, and development practices. When we're planning the architecture, we'll spend a lot of time in the exploration mindset, but this mindset is also useful when working with stakeholders.

Make It Real

As you learned in [Make the Architecture Tangible, on page 17](#), ideas are great but if you can't transfer them from your brain into someone else's brain, then your ideas are useless. Making ideas real gives us a way to share them but also provides an opportunity for testing an idea. In the *make mindset* we turn our design concepts into real-world artifacts.

The most common ways we make architecture real is by creating models. Making goes way beyond box and line diagrams. You can make the architecture real by building prototypes, writing documents, crunching numbers, and a variety of other approaches.

The make mindset is useful for communicating our plans. We'll also make the architecture real as we build the system—for example, by organizing our

code so that it's possible to *see* module structures in the architecture. Making is also an excellent way to push your team out of analysis paralysis.

Evaluate Fit

How do you know if a design decision will solve the problem? When we embrace the *evaluate mindset*, we determine the fitness of our design decisions relative to our current understanding.

Evaluation is not an all or none proposition. We can evaluate all or part of the architecture, even only a single model, concept, or idea. The most common approach is to walk through a piece of the architecture with different scenarios, but we can also test design decisions directly by running experiments or examining the risks surrounding a decision.

The evaluate mindset comes in handy when we want to verify the planned or built architecture, but this is only the beginning. This mindset will help us inspect anything we make and decide whether that artifact is serving our need.

Using design mindsets requires a process with a tight feedback loop so we can quickly move from one mindset to the next. In the next section, we'll learn how to use a simple, iterative approach to help us choose and use design mindsets.



Get Your Hands Dirty: Understand, Explore, Make, Evaluate

The four design thinking mindsets reflect how people solve problems. Even without training in design thinking, you have probably used these mindsets before. What are some examples of how you embraced each of these mindsets so far in your software development career? Try to name at least two examples of how you worked in each design mindset.

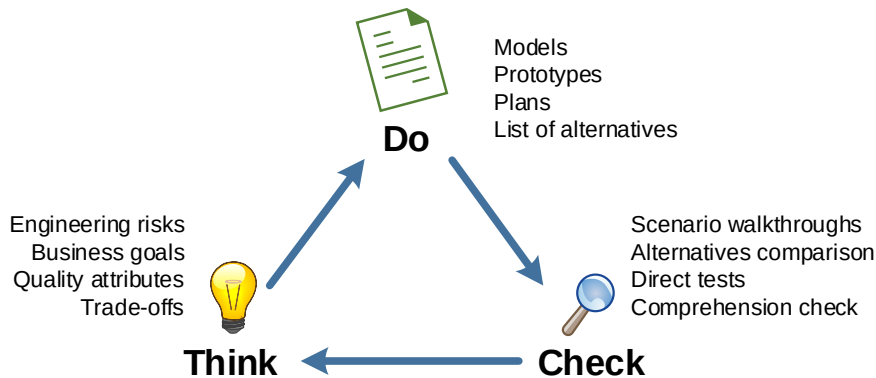
Here are some things to think about:

- When have you worked with people to *understand* a problem? Did you follow a particular method?
- How have you collaborated with others to *explore* ideas and generated alternatives?
- Looking beyond code, how do the things you *make* change how you interact with stakeholders and teammates?
- How do you *evaluate* your designs? What techniques have you used to test solution hypotheses?

Think, Do, Check

Every day, for as long as we work on a software system, we learn interesting things about the software. Every new thing we learn might force the architecture to evolve to reconcile the new information. For us to keep tabs on this ever changing landscape, we need a design approach with a tight feedback loop that gives us the opportunity to change our mindsets often.

There are three steps in our approach: think, do, and check. Each iteration of the think-do-check cycle focuses on a particular design mindset.



Iterate to Learn

An iteration can be as brief as a few minutes or as long as a few days. We prefer shorter cycles over longer ones, but sometimes more time is required for in-depth research. Every iteration follows the same steps, though the execution will vary depending on the design mindset we adopt.

Think What do we hope to learn? What questions do we need answered? What are our top risks? Thinking involves creating a plan to learn what we need to answer specific questions or reduce risks.

Do Execute the plan. Create something tangible that quickly and cheaply uncovers information needed to check our thinking and share our ideas.

Check Critically examine what we accomplished during the *do* step so we can decide our next move. The insights coming out of the *check* step tell us what to do next. Repeat at the *think* step.

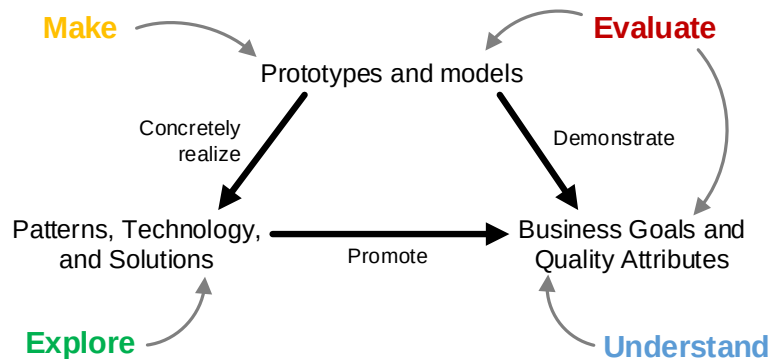
A software system is never finished; it is only released. Since software is never done, our design approach has no end. Anytime you need to revisit

some aspect of the architecture, whether it's to evolve an existing design or create something new, the same approach applies.

Adopt Mindsets in Any Order

Think of the four design mindsets like four tool boxes, each containing tools tuned for a particular type of design work. When the need arises, embrace the mindset required to learn more about the world or reduce a risk.

In the understand mindset, we'll focus on stakeholders needs and how to specify those needs as requirements. In the explore mindset, we'll brainstorm ways to solve the problem as we understand it by looking at patterns, technology, and other solutions. In the make mindset, we'll model the system so we have something concrete to reason about and share. In the evaluate mindset, we'll put our models and requirements to the test.



Mindsets shift frequently and quickly. During a single conversation, we might change mindsets several times. During a workshop, we will create situations that force participants to adopt new mindsets so that we can arrive at a desirable outcome. You'll see an example of this in [Chapter 9, Host an Architecture Design Studio, on page 113](#).

Experienced architects are often unaware that they attack architecture from varying perspectives. They fly by instinct and adopt different mindsets intuitively (thanks to years of practice). Awareness of the four design mindsets gives us new techniques for getting out of a rut. If you get stuck, choose a new mindset to get yourself unstuck.

Plot Your Course: Think-Do-Check and Mindsets in Action

Let's see a concrete example of how the think-do-check cycle and design mindsets might play out in practice. Say a stakeholder just gave us a new

constraint and this increases the risk that the architecture will be unable to satisfy performance requirements.

- *Think.* We know system performance is important, but we don't know what it means to have *good* performance. Since we need information about the problem, let's adopt the *understand* mindset. Looking through some practices in [Chapter 14, Activities to Understand the Problem, on page 191](#), we decide to capture quality attribute scenarios.
- *Do.* We brainstorm some performance scenarios and record them in a document.
- *Check.* The team and stakeholders review the scenarios and provide feedback.

Based on what we learned, new risks arise. Can we achieve the performance quality attribute scenarios given the new constraint?

- *Think.* Since we need to verify that our decisions promote a specific quality attribute let's adopt the *evaluate* mindset. We plan an experiment so we can directly test the constraint's impact on performance.
- *Do.* We write some simple scripts to drive existing parts of the software system and collect data. We run the experiment.
- *Check.* With data in hand, we examine the results and conclude that the new constraint negatively impacts performance but only by a few 100 milliseconds.

We think we've done a thorough job, but performance is funny. Hurting performance might not be a big deal until it degrades too much. We need to share these results with our stakeholders and discuss the implications of the new constraint.

- *Think.* Since making ideas tangible facilitates communication, we'll adopt the *make* mindset and create a simple prototype. We want stakeholders to experience the impact of the new constraint. Graphs aren't enough.
- *Do.* We develop a throwaway prototype that demonstrates the application workflow and simulates different assumptions about performance.
- *Check.* We give the prototype to our stakeholders and explain why the performance of the system was impacted. On paper, a few 100 milliseconds is tiny, but experiencing the slowdown firsthand shows that this dip in performance isn't acceptable.

The prototype helped our stakeholders learn something about the problem nobody knew was important until now. Next, we'll adopt the *understand* mindset and refine our new requirements. We check our understanding a few minutes in the same meeting by selecting the *explore* mindset. And the cycle continues.

The think-do-check cycle is extremely flexible. How you use it depends on the complexity and size of the system, your team's size and skills, and your experience with having simultaneous design initiatives in flight.

Next Up

Design thinking gives us a way to connect the highly technical world of software development with the humans affected by the software we build. The four HART principles are the means by which we'll give our software heart (pun 100 percent intended). Design mindsets are the way we'll decide what needs to be done to help our stakeholders.

Now that we've covered the theory, it's time to get down to business.

Since the invention of software, we have debated how much architecture design should happen up front and how much can emerge as we implement a solution. Like any discussion about extremes, the real answer lies somewhere in the middle. In the next chapter, you'll see how to define a design strategy appropriate to your situation and choose design mindsets by considering the risks in the software system.

Part II

Architecture Design Fundamentals

In Part I you learned about design thinking principles and mindsets. In Part II you'll see how to put those principles and mindsets into practice to design software architectures.

Understand

Explore

Evaluate

Make

CHAPTER 3

Devise a Design Strategy

Designing software architecture always feels a bit chaotic. Despite the swirl of uncertainty surrounding every software system, it's important to have a plan. With a solid design strategy, we can feel our way through the mists of uncertainty.

Design thinking is perfect for finding solutions to complex problems. Instead of trying to solve the problem perfectly on the first try, design thinking emphasizes learning and experimentation. Testing an architecture might be impossible without implementing it, but it may be possible to verify pieces of the architecture incrementally as we design them. We can use design mindsets and the think-do-check cycle to decide where to focus our attention next.

In [Chapter 2, *Design Thinking Fundamentals*, on page 15](#) you learned the basic rules of design thinking and how to use design mindsets. In this chapter, you'll learn how to choose design mindsets as part of a broader design strategy by thinking about the risks in the software system.

Find a Design That Satisfices

In a rational world, we'd fully define the problem before designing a perfect architecture to solve it. Too bad we don't live in a perfect, rational world. In [The Sciences of the Artificial \[Sim96\]](#), Herbert Simon coined the term *bounded rationality* to describe the theoretical barrier created by limits in time, money, skills, and knowledge that make rational design challenging for complex problems such as software architecture.

Instead of rationally seeking an optimal design, our goal is to find an architecture that *satisfices*. A satisficing design is both satisfactory and sufficient—good enough—for our needs.

Instead of thinking of software architecture as a design optimization problem, we'll look for a satisficing design by emphasizing the following activities.

Treat solutions as experiments. Architects are not omniscient sages of technology who know all things. Think of every potential solution as an experiment to be validated. The sooner, faster, and cheaper we can validate (or invalidate) our hypotheses, the sooner we'll find the right combination of structures that will help our stakeholders and the sooner our stakeholders will gain value from our designs.

Focus on reducing risks. Value is only one variable that must be considered. Architecture is the foundation of the software system. If it fails, then everything fails. Architects must constantly worry about what could go wrong and design for these scenarios. We can use risk to help us decide what to design next.

Work to simplify problems. Simple problems often have simple solutions. There are many ways to simplify the problem. Reducing the number of stakeholders will decrease the variety of competing perspectives influencing the system. Adding or removing constraints, or focusing on a subset of the problem can reduce complexity. Identifying the *routine problems* makes it easier to focus on redesign. Routine problems have a known solution so we can start with pattern catalogs and apply our collective experience when exploring solutions.

Iterate quickly to learn quickly. The faster we learn, the more we can explore, and the greater confidence we'll have in our solutions. If we're wrong, let's find out as quickly as possible. Failing fast means learning fast. Favor short, tight design iterations with concrete outcomes over longer design iterations that only focus on abstract goals.

Think about the problem and solution at the same time. In *Notes on the Synthesis of Form* [Ale64], Christopher Alexander shows how problems are always defined with a solution in mind. The boundary around a problem is created by the solutions that could potentially solve it. To understand the problem, we must explore solutions. To do a better job of exploring solutions, we must improve our understanding of the problem. Designing software architecture requires us to think about problems and solutions simultaneously. Writing some code early in the design process is one strategy for dealing with the reciprocal relationship between problems and solutions.

Avoiding rational design does not mean we suddenly become irrational architects. Architecture is the foundation of every software system, so we still

need do some design work up front. We'll need to decide how much of the architecture we'll design up front and how much we'll allow to emerge over time. Selecting a design strategy early in a software system's life tells our team how we want to grow the architecture and instills confidence in our stakeholders.

Decide How Much to Design Up Front

Architecture is a necessary investment. Every software system has an architecture. One way or another, you will spend time designing it. If we spend time designing architecture up front, then we'll reduce the cost of future rework. Of course, time spent planning architecture also delays implementation thus potentially delaying value for stakeholders. If we spend no time up front on architecture, then we are more likely to make changes to the architecture after developing parts of the software system.

Depending on the software system's size and requirements variability, every software systems has a *design sweet spot*, an optimal amount of time to spend designing architecture before diving into implementation.

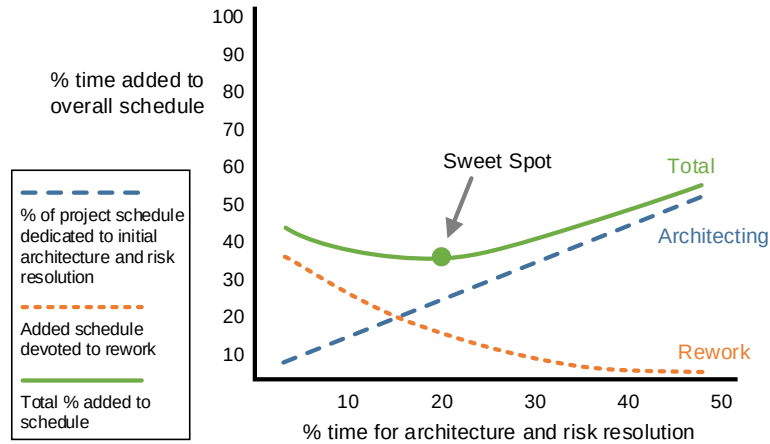
Find the Design Sweet Spot

In [Architecting: How Much and When? \[BWO10\]](#), Barry Boehm shows us that the combination of development time, architecture planning, and rework are major contributing factors to the overall project schedule. Rework includes activities such as fixing design defects, rewriting code, and undoing mistakes. To find the sweet spot, we must account for both design costs and the inevitable rework required to complete the software system.

$$\begin{array}{l}
 \text{Development Time} \\
 \text{Architecture and Risk Reduction Time} \\
 + \text{ Rework Time (fixing defects, rewrites, mistakes)} \\
 \hline
 \text{Total Project Time}
 \end{array}$$

Note: Time spent on architecture can speed up development and reduce rework!

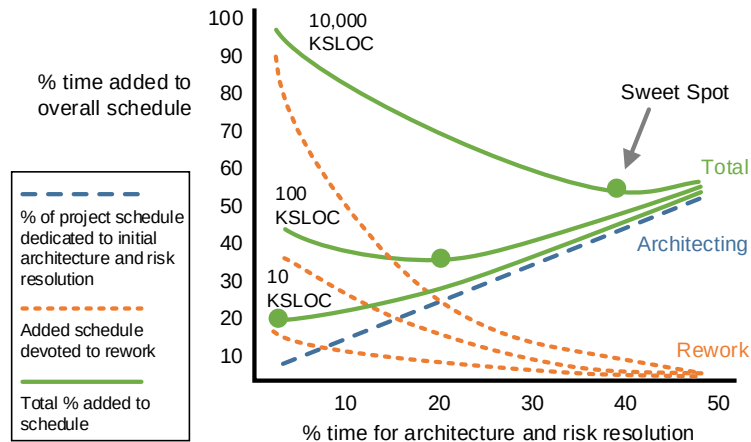
The architecture sweet spot is highly dependent on the size, requirements volatility, and complexity of the software you're developing. The [graph on page 30](#) from [Boehm's data \[BWO10\]](#) shows how the amount of rework decreases as more time is invested in architecture planning. The solid green



line represents the sum of architecture design effort (dashed blue line) and the cost of rework (dotted orange line).

In this case, spending less than about 20 percent of the original project schedule on architecture has a diminishing return. While the amount of rework decreases with a greater investment in architecture, the total project schedule increases. Similarly, spending less time on architecture raises the amount of rework, creating a longer overall project timeline.

In the same research, Boehm also shows how the sweet spot moves depending on the estimated size of the software system. Use this data to decide if the amount of time you plan to dedicate to up-front architecture design is in the right range.



This graph is packed with important implications, so let's break down the essential points.

The bigger the software system, the greater the benefits you'll reap from up-front architecture design. From Boehm's research, spending upward of 37 percent of the total estimated development schedule on architecture design for large (10,000 KSLOC, where a KSLOC is 1,000 equivalent source lines of code) software systems would be a wise decision.

Smaller (10 KSLOC) software systems gain little from up-front architecture planning. From Boehm's research, as little as 5 percent of the total estimated development schedule might be spent on up-front architecture design. In some cases, it can be faster to rewrite a small software system than to spend significant time designing architecture up front.

Expect violent thrashing if you invest little into architecture design. Avoiding up-front architecture design in smaller software systems might be more cost effective and result in a shorter overall schedule, but rework will still be necessary. Be prepared for this and account for design churn in your plans. Bigger systems will experience even more thrashing with less architecture design up front.

The more you invest in architecture, the less rework will be required. Planning architecture helps avoid mistakes. If you favor predictable project schedules over schedule efficiency, you will benefit from more planning up front, even on smaller software systems. Up-front planning is a must in larger software systems.

Size is a nice predictor since it's easy to measure and estimate, but [many teams also use complexity \[WNA13\]](#) to decide how much architecture design work might be required. Large systems can be complex, but not all complex systems are large. If a solution is routine, then your team might get away with less up-front planning even when the system is large.

Another factor to consider is requirements volatility. Changes to architecturally significant requirements can invalidate the best-laid plans. If you anticipate a high degree of change, delay making binding decisions and focus on using lighter-weight design and documentation methods.

Example: Impact of Architecture on Total Schedule

Say we are developing a software system with an estimated size of 100 KSLOC and an initial development schedule of 100 days. Per Boehm's data, if we spent 5 percent of our time on architecture, then the total project schedule would increase by about 43 percent. Had we spent a little more time on architecture, say 17 percent of the estimated development schedule, then the total project schedule would have increased by only 38 percent.

Of course, more architecture does not always mean we'll have a shorter schedule. If we spent a third of the estimated development time on architecture, we'd increase the total development schedule. While we'd spend significantly less time on rework, the extra time dedicated to architecture doesn't pay off since the overall project schedule increases by about 40 percent.

Here are the numbers assuming our estimated 100-day initial development schedule:

Days Spent on Architecture	Days Spent on Rework	Days in Total Schedule
5	38	143
17	21	138
33	7	140

The Constructive Systems Engineering Model (COSYSMO) includes Boehm's findings. I suggest you try the COSYSMO and COCOMO II tools with your project data.¹

Thanks to Boehm's work, we have a general idea for how much time to spend on architecture design, but we still don't know when to do the design work or when to adopt the different design mindsets introduced in [Chapter 2, Design Thinking Fundamentals, on page 15](#). It turns out Boehm also has an answer for this (yes, he is that awesome). In [Using Risk to Balance Agile and Plan-Driven Methods \[BT03\]](#), Barry Boehm and Richard Turner propose using risk to decide when to focus on architecture. If we think about risk in the right ways, we can also use it to determine what to design and how to involve stakeholders in the design process.

Let Risk Be Your Guide

Shortly after the first stakeholder meeting on a new software project, I always feel a giant pit grow in my stomach. If I didn't have that feeling, then I would be worried. Software worth building always has risks. You *should* feel a bit uncomfortable at the start of a new project. After all, if you knew everything at the outset and had no questions about what you were going to build, then why would an architect be needed?

We can use that slight sinking feeling in our guts to our advantage. Risk is an excellent indicator of what might prevent us from succeeding. To harness the power of our guts, write down all the things that worry you about the software

1. http://sunset.usc.edu/csse/research/cocomoii/cocomo_main.html

system. Next, prioritize the items on your list so that the things likely to cause the most trouble are highest on the list. Finally, pick one of the top things you're worried about and choose a design mindset to reduce the risk.

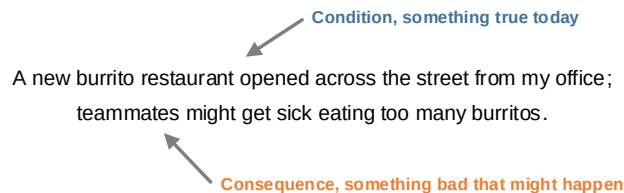
Gut feelings are a good start, but guts alone don't give us much to go on. I don't know about you, but I hate going to my boss with gut feelings. *Hi, Will, so I'm not sure if it was the burrito I had for lunch or what, but I've got a bad feeling about how the system is going to scale as we add data.* Gut feelings tell us there is something wrong, but we can do a much better job than just listening to our lower intestines.

Identify Conditions and Consequences

A *risk* is something bad that might happen in the future. If it already happened, then it's called a problem. If it starts with *what if...*, then it's pure speculation. We could play *what if...* all day and never get any closer to designing a useful architecture. Instead of speculating, we'll use what we know about the architecture today to help us decide what to design next.

There are two parts to every risk. The *condition* is a fact about the world that is currently true. The *consequence* is something bad that might happen in the future as a direct result of the condition. We record risk statements using the simple template from [A Construct for Describing Software Development Risks \[Glu94\]](#), *<Condition>; might <Consequence>*.

Here's a risk statement in the condition-consequence format.



There are many ways we could reduce or remove this risk:

- Reduce the probability. Hire catering to bring in food a few days a week, and hold an info session on responsible meal portions.
- Reduce the impact. Keep a supply of antacids in the office.
- Push out the time frame of the risk. Schedule meetings at lunch so people can eat burritos only for dinner.
- Remove the condition. Move the office to a new location. Change the shift schedule so that everyone works at night when the new restaurant is closed.

- Accept it and do nothing. Sometimes people eat too many burritos (*mmm... guacamole*). We'll deal with the repercussions if the risk converts into a problem.

Knowing the condition and the consequence creates hooks for deciding what to do about the risk. Compare our *<Condition>; might <Consequence>* risk with these other, less effective risk statements:

Bad Risk Statement	Why It's Bad
New burrito restaurant opened.	So what? The negative impact is unclear. In fact, this sounds great. We love burritos!
The team might overindulge in burritos.	This sounds bad, but why should we be worried about this now?
Eating too many burritos can make you sick.	True, but what does this have to do with my team?
If a teammate eats too many burritos, he or she will get sick.	And if a radioactive meteorite falls on the office, we'll get sick too. What led you to be worried about burritos?

OK, enough about burritos. Let's get back to software design.

Use Risk to Choose a Design Mindset

Software architecture design is an exercise in risk reduction. Every time you think, *I've got a bad feeling about this*, it's a sign there's a risk looming nearby. If you can pick a condition and consequence that captures the essence of your bad feeling, then we can use that information to guide our design activities.

Here are some example risks from a past project I worked on and the design activities the team used to reduce the risk:

The Model Training service was originally built for a different purpose; might overload it with new requests.

Design Mindsets: Understand, Evaluate

What we did: Talked to the team who built the Model Training service to understand scalability, ran experiments to measure throughput.

Data processing is time consuming and resource intensive; might not be able to finish processing jobs without failures.

Design Mindset: Explore

What we did: Brainstormed approaches for promoting reliability, researched job scheduling patterns, and sketched alternative designs that might reduce processing time.

A lot of data is needed to train a statistically significant model; might not be profitable due to data storage costs at scale.

Design Mindset: Make

What we did: Created a cost estimation model. The model demonstrated pros and cons of different design options to stakeholders. Pushed out the risk's time frame by changing backlog priorities.

Stored data may contain sensitive customer information; might require stricter data isolation than we can provide.

Design Mindset: Evaluate

What we did: Rated available compute platforms based on how well each met our needs.

Engineering risks help us decide what to design. Design mindsets help us devise a strategy to decrease the risk. When facing a risk that must be reduced, first decide which parts of the risk you can address—condition, impact, probability, or time frame. Next, choose a design mindset. Here are some questions to help decide which design mindset might be appropriate:

Try...	If...
Understand Mindset	The risk is about the problem. Do you need a deeper understanding of stakeholders or other system actors?
Explore Mindset	The risk about the solution. Have you seen enough solution alternatives?
Make Mindset	The risk is about communication. Do stakeholders fully understand design concepts at play and can they see the architecture?
Evaluate Mindset	The risk involves a design decision or the design's overall fit. Do we need to make a design decision?

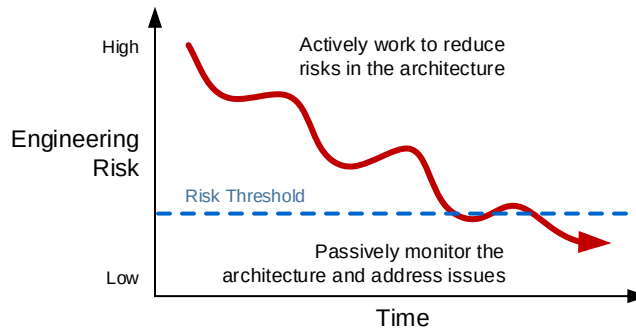
Risks are the GPS for our design process. They tell us where we are, where we're going, and how much we have left to design. With each loop through the think-do-check cycle [discussed on page 21](#), think about the risks and use them to decide what to do next.

Shift to Passive Design Once Risks Are Reduced

In [*Just Enough Software Architecture: A Risk-Driven Approach* \[Fai10\]](#), George Fairbanks tells us that architects should work to reduce technical risks to the point where architecture is no longer the biggest source of risk in the

system. Once we have reduced enough of the architectural risks, whatever that means for the software system, then our time is better spent elsewhere.

Once architecture is no longer the greatest source of risk in the system, shift from *active design* to *passive design*, as shown in the graph. With active design, we drive the design process in the pursuit of reducing risk. With passive design, we observe the architecture as it manifests in the working system and take corrective actions as necessary.



Don't be tricked by the name. There is still a lot for an architect to do in passive design mode. We might correct errant documentation or improve missing documents. We'll make minor adjustments to the architecture as new information emerges. We'll teach our teammates about the architecture by pairing and reviewing code. Most importantly, we're on the front line in the fight against architectural erosion and other issues, [discussed on page 172](#).

Even with careful vigilance, architecture could reemerge as a significant risk. New risks might emerge. The implemented system could drift too far from our plans. We could learn that our assumptions about the world are wrong or that the world around us changed. When these things happen, switch back to active design and adjust the architecture based on the new realities.

Now that you know how to use risk to help you decide what to design, let's pull together everything you've learned in this chapter by ceating an architecture design plan.

Create a Design Plan

Design plans outline a general strategy for how the team will spend their time on architecture. Will we do more analysis up front? Are we expecting change later? When do we start writing code? A good design plan sets expectations and explains these details.

A design plan doesn't have to be a formal schedule, but you do need to put some thought into it. Here are a few things every design plan should include: Capture your plan in a lightweight document such as an inception deck [described on page 269](#).

Stopping conditions for design Will you time-box up-front design work, or will you reduce risks no matter how long it takes? Will you do minimal up-front design before starting to write code, or do you want more of the architecture laid out? Can implementation start piecemeal, or do some areas need to start together? There is no single right answer. Stopping conditions depend heavily on the team, stakeholders, and project context.

Required design artifacts Tell everyone how you plan to document the architecture before starting. Are you OK with pictures of whiteboards, or do you need a more traditional document? Does your team use specific templates? Where should design artifacts be stored?

Time line Describe the key design milestones within the project schedule. Many large projects have a dedicated elaboration phase for gathering requirements and exploring architecture. Smaller projects or continuously maintained software systems might regularly schedule design spikes.

At a minimum, the time line should include milestones for reviewing architecturally significant requirements, reviewing draft designs, and conducting evaluations. Also, include any major workshops with stakeholders. Call out when you think implementation will start and what is in scope for early implementation.

Top risks Since we are using a risk-driven design approach, include the top risks in the design plan as context. Revisit your risk list throughout the software system's life, especially during any up-front architecture design.

Notional architecture design Start with a potential solution. Recall that we need to think about the solution to help us define the problem. A notional architecture can be a lightweight sketch, just enough to communicate the essence of your initial design thoughts.

The amount of time spent on design could be hours, days, or even months depending on the software system. No matter your time horizon, if we use the four principles of design thinking [covered on page 15](#) and focus on finding a satisficing solution [discussed on page 27](#), then we should arrive at a working solution by the time we need it.

Project Lionheart: The Story So Far...

Next week you'll go on site to gather requirements from the mayor and other stakeholders. We have a fixed final due date in about six months. We need to focus on value up front and deliver as fast as possible. It also sounds like the core functional requirements will be based on an existing process, so there is a low probability of requirements churn.

The solution seems like a classic data-driven web application with some search features. Based on the mayor's description, security and privacy could be a key concern. We also know that the city's IT department will take over the software system after us. That group may impose some unique constraints.

You send Mayor van Damme an agenda for the on-site visit. Our biggest risks right now can be addressed by digging for information, so for now you'll focus on getting to know our stakeholders. We think we can get away with very little up-front design and want to concentrate on delivering value faster, even if that means rewriting parts of the application later. The team wants to start writing code immediately after a two-week design spike.

Next Up

One way or another, you'll pay for your architecture either through up-front design or downstream rework. In this chapter, you learned how to use risk to plan our design activities. Risks can help us decide how much work to do up-front. We can also use risk to decide which design mindsets to adopt.

One of the first risks many teams face on a new software system, even if it's only new to you, is understanding who the software is meant to help. In the next chapter, you'll learn how to embrace the understand mindset by developing empathy for the humans who benefit from the software you develop. When you can walk in your stakeholders' shoes, you'll get a deeper understanding of their actual needs. Understand stakeholders' real needs, and you improve your chance of solving the right problem.

Empathize with Stakeholders

Knowing what problem to solve is sometimes easier said than done. Since we create software to help people, we must understand the people whose lives will be affected by the software we make to understand the problem thoroughly. The better we empathize with their needs, the better we'll see and understand the real problems that need to be solved.

We call people with an interest or concern in our software *stakeholders*. It's the architect's job to identify stakeholders and understand their needs. Our stakeholders' expectations for the system will directly or indirectly influence how we design it.

Empathy is the engine that drives design. When you empathize with the people affected by your software, then you'll make better software. In this chapter, you'll learn how to decide who to talk to about the problem you're solving and what you need to learn from them to start designing the architecture.

Talk to the Right People

Stakeholders usually, but not always, have a business interest in the software. They might pay for the software or directly profit from it. Users are important stakeholders but so too are the people who build and maintain the system. Other people might not even realize how our software might affect them, but it's sometimes necessary to consider their concerns as well.

In the wild, stakeholders rarely travel alone. We use the term *stakeholder group* to highlight this fact. Working with groups is different than working with individuals. Two people from the same stakeholder group can provide inconsistent or conflicting information. We must work to understand the whole group's concerns and sometimes even help them reach consensus.

Here are some stakeholders for the Project Lionheart Case Study, [introduced on page 14](#). In this picture, the icons represent specific people or roles while speech bubbles show stakeholders' thoughts and feelings.

Some Project Lionheart Stakeholders



Stakeholders are interested in what we're building and will influence the architecture we design. Since we'll want to invite these people to future design workshops, we should find out who they are. Enter the stakeholder map.



Bett says:

It's About the Customer

By Bett Bollhoefer, software architect at General Electric

Architecture is about the customer. If I create an architecture that doesn't give value to the customer, I am wasting my time. When I talk to customers, I often hear horror stories about how their current systems were developed by someone in an ivory tower who didn't understand them or their work. How do I make sure I am bringing value to customers through my architecture?

My answer is to use a customer-centric design process. I start with who the customer is and what they want to do. I divide the system into tasks the customer performs. For each task, I find out how they start it and where they run into issues.

You might be thinking, "This doesn't sound like architecture—it sounds like user experience!" Yes, it is, but many UX designers don't understand the technical aspects of the system well enough to determine the architecture. My process goes beyond the

surface to ensure the deep structures support the customer's values. I call it *Customer Experience Architecture*.

Step 1: Determine what matters to the customer, including their functional requirements and quality attributes, by watching how they do the task in their natural environment and asking lots of *why*'s.

Step 2: Design the system around the customer's needs and document it in a prototype. The prototype should be as interactive as possible, not just a flowchart.

Step 3: Review the prototype as early as possible with the customer. Make sure they really understand what is changing in the new system and how it will impact them.

Step 4: Revise the architecture based on feedback from the customer's review.

Using these four steps, you can create value for your customer through your architecture and become their hero, or at least not the person in the ivory tower who is ruining their life.

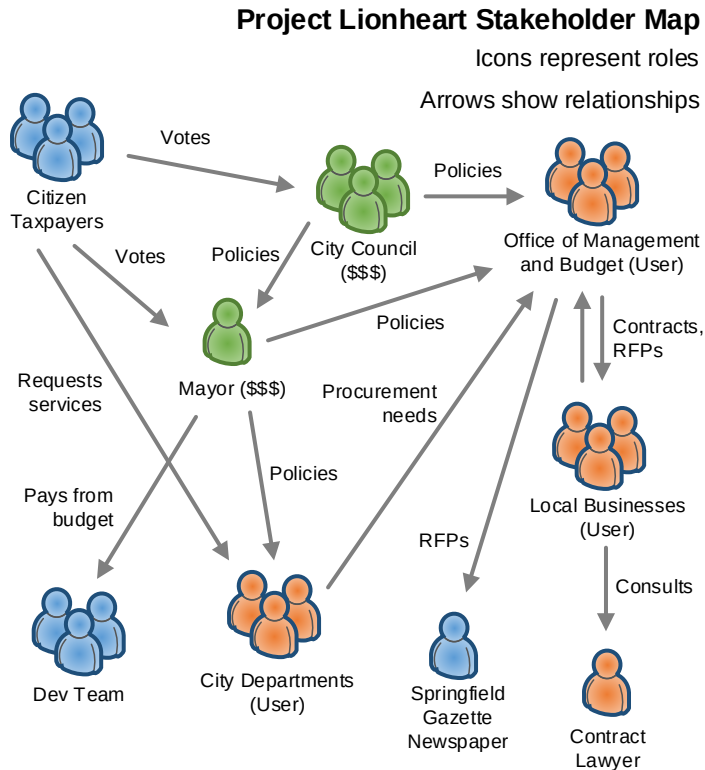
Create a Stakeholder Map

A *stakeholder map* is a network diagram showing all the people involved with or affected by the proposed software system. Stakeholder maps are ideal for visualizing relationships and interactions among people. They also give you a snapshot of what motivates different stakeholders. Use stakeholder maps to decide who the most important people are to talk to about their concerns.

Every time I create a stakeholder map, I'm surprised to see how many people I might touch with the software I build. There's a partial stakeholder map for Project [Lionheart on page 42](#).

There are several stakeholders not shown on this diagram for the sake of simplicity. Additional stakeholders include IT vendors we might have to collaborate with, the Chamber of Commerce (or other lobbying organizations), the deputy mayor, and various community groups who receive services from the city. The *city departments* stakeholder can be made more precise by dividing it into the board of education, parks and recreation, public works, sanitation, and so on. If these groups have a similar stake in the system, then they could be lumped together as shown. As a rule of thumb, it's best to be as specific as possible.

Step back and look at the stakeholder map after you've created it. Who is paying for the software? Who is using it? Are there network hubs with many incoming or outgoing arrows? Are there stakeholders with potential conflicts of interest? These people are all excellent candidates for interviews and further research.



When I look at the Project Lionheart stakeholder map, I see a few interesting areas that we should investigate further.

1. Mayor van Damme hired us and we report to him, but the Office of Management and Budget receives policy direction from both the mayor and city council.
2. Our software will affect many city departments, but we won't be able to talk to all of them. We should identify a few representative stakeholders and carefully validate our findings with the larger group.
3. Some local businesses rely on lawyers to navigate the Request for Proposal process. Different interaction patterns for the potential software might exist, which could influence the architecture.
4. The Office of Management and Budget (OMB) sits at the center of several key user interactions, but they aren't paying the bills. We should talk to the OMB directly. It's possible that the mayor and city council have budgeted for a system that does not solve the OMB's real problems.

You can build a stakeholder map by yourself, but it's more fun to create them in groups. The steps for this activity are outlined in [Activity 10, Stakeholder Map, on page 221](#).

 Get Your Hands Dirty: Create a Stakeholder Map

Pick an open source project you use or contribute to and create a stakeholder map for it. Take a picture of it and share it on this book's forum.¹ Here are some things to think about:

- Is there an organization that oversees or funds the project? Are there sub-groups within the organization who might have different vested interests?
- Who are the biggest contributors to the project?
- How is the project licensed? Who benefits from the choice of license?
- Who uses the project? What problems are they trying to solve?

Discover the Business Goals

Every software system is built to serve some fundamental purpose. *Business goals* describe what stakeholders hope to accomplish with the software. Business goals also seed conversations about quality attributes, trade-offs, and technical debt.

Business goals are a primary architectural driver and help prioritize competing concerns. The better everyone understands stakeholders' needs, the better you'll be able to help them. Here's a summary of common business goal categories:

Who wants it	What they want
Individuals	Increase wealth, power, reputation, personal enjoyment, or knowledge
Organizations	Increase revenue, maximize profits, grow the business, become a market leader, improve stability, enter a new market, beat a competitor
Employees	Interesting and meaningful work, increase knowledge, help users, become recognized as an expert
Development Team	Improve specific quality attributes, reduce costs, add new features, implement a standard, improve time-to-market
Nations, governments	Security, civic welfare, social responsibility, legal compliance

1. <http://pragprog.com/book/mkdsa/design-it>

Capture business goals as simple need-based statements that explain what stakeholders will get from the software system.

Record Business Goal Statements

Great business goal statements are measurable and have clear success criteria. Human-centered business goals allow your team to understand the people you are ultimately serving through the software you build.

Good business goal statements include three things:

Subject A specific person or role. If the stakeholder or group has a name, use it. *United Hamster Trainers Union* is better than *union groups*.

Outcome Express the stakeholder's need as a measurable outcome. How does the world change if the system is successful? You will design an architecture to achieve this outcome. For example, maybe the United Hamster Trainers Union needs a way to help members stay in touch with one another.

Context Context shares an insight about a stakeholder's need and helps build empathy. Ideally context is insightful and not completely obvious. For example, knowing that the United Hamster Trainers Union's most important annual meeting has over 5 million members attending virtually creates deeper understanding about the previously discussed outcome.

There are some business goals for Project Lionheart in the [table on page 45](#). Putting business goal statements in a slick-looking table sometimes makes them easier to read.

Most systems only have only three to five business goals. More than this and the goals become confusing and difficult to remember. When working with many stakeholders, it's useful to record goals' relative importance. A simple *must have* or *nice to have* designation is good enough for this purpose.

Help Stakeholders Share Their Business Goals

Stakeholders usually know what they want, but many stakeholders find it difficult to articulate their needs as measurable statements. Every architect should have a few simple templates in their toolbox to help stakeholders find their voice. The point of view (POV) [mad lib on page 45](#) is a fun alternative that is similar to a user story but describes the value expected from the whole system instead of functionality. Other business goal formats are described in [Activity 8, Point-of-View Mad Lib, on page 215](#).

Stakeholder	Goal	Context
Mayor van Damme	Reduce procurement costs by 30%	Avoid making budget cuts to education or other essential services in an election year.
Mayor van Damme	Improve city engagement with local businesses, measured by the number of applications from first-time local businesses, percentage of overall RFPs won by local businesses.	Improve the local economy by ensuring local businesses win city contracts.
Office of Management and Business	Cut the time required to publish a new RFP in half.	Improves services across the city and reduces costs at the same time. Citizens suffer when city services go unfunded. Think: <i>No toilet paper at the girls' basketball game or not enough hypodermic needles for emergency medical crews.</i>
Office of Management and Business	Review historical procurement data for the past 10 years.	Businesses behave similarly over time and historic data gives the city a leg up when reviewing contract bids.

Mayor van Damme needs to reduce procurement costs by 30%

(stakeholder)

(stakeholder's need)

because he wants to avoid cutting essential department funding.

(context)

Collaborate closely with your product manager or other business-focused stakeholders to identify the system's business goals. They can usually describe a system's business goals without breaking a sweat. Be prepared to help stakeholders or product managers if they struggle to articulate business goals, but remember they own the business goals.